



Multi-Architecture Images

Learn what multi-architectural images are and how they provide support across different platforms and architectures.

We'll cover the following

- Multi-architecture images to the rescue
 - Internal working of multi-architectural images
 - Theory
 - Implementation
 - Linux on x64 example
 - Windows on x64 example
 - Docker manifest
 - Playground

One of the best things about Docker is its simplicity. However, as technologies grow, things get more complex. This happened for Docker when it started supporting multiple different platforms and architectures, such as Windows and Linux, on variations of ARM, x64, PowerPC, and s390x. All of a sudden, popular images had versions for different platforms and architectures. As developers and operators, we had to make sure we were pulling the correct version for the platform and architecture we were using. [Got any feedback? Get in touch with us.](#)

Docker experience.



Note: We're using the term "architecture" to refer to CPU architecture, such as x64 and ARM. We use the term "platform" to refer to either the OS (Linux or Windows) or the combination of OS and architecture.



Multi-architecture images to the rescue

Fortunately, Docker and Docker Hub have a slick way of supporting multi-arch images. This means a single image, such as `golang:latest`, can have an image for Linux on x64, Linux on PowerPC, Windows x64, Linux on different versions of ARM, and more. To be clear, we're talking about a single image tag supporting multiple platforms and architectures. We'll see it in action in a second, but it means you can run a simple `docker image pull golang:latest` from any platform or architecture and Docker will pull the correct image for your platform and architecture.

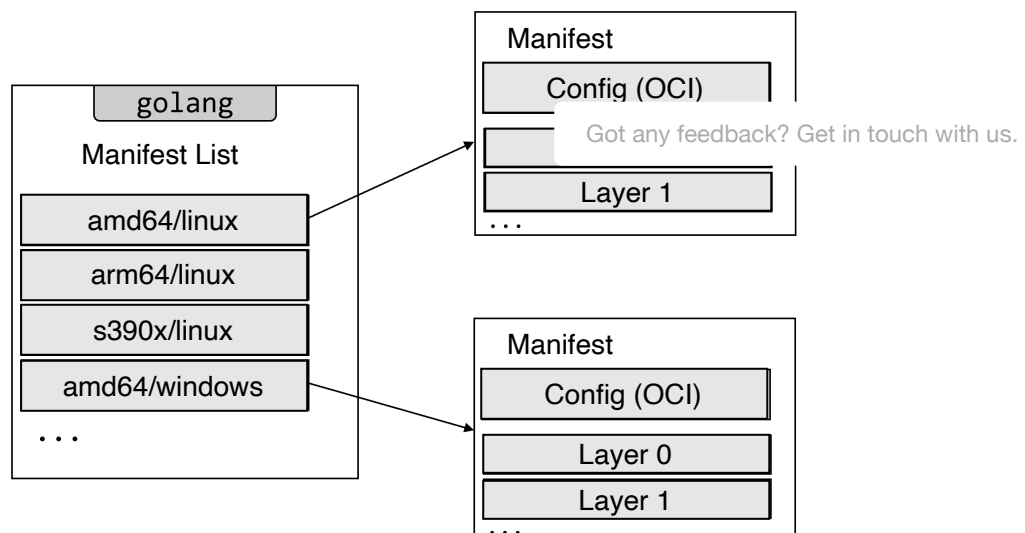
Internal working of multi-architectural images

To make this happen, the Registry API supports two important constructs:

- **Manifest lists**
- **Manifests**

The **manifest list** is exactly what it sounds like: a list of architectures supported by a particular image tag. Each supported architecture then has its own **manifest** that details the layers that make it up.

The figure below uses the official `golang` image as an example. On the left is the **manifest list** with entries for each architecture the image supports. The



Theory

> Let's look at the theory before seeing it in action.

Assume you are running Docker on a Raspberry Pi (Linux running on ARM architecture). When you pull an image, your Docker client makes the relevant calls to the Docker Registry API exposed by Docker Hub. If a **manifest list** exists for the image, it will be parsed to see if an entry exists for Linux on ARM. If an ARM entry exists, the **manifest** for that image is retrieved and parsed for the crypto ID's of the layers that make up the image. Each layer is then pulled from Docker Hub.

Implementation

The following examples show how this works by starting a new container from the official **golang** image and running the **go version** command inside the container. The output of the **go version** command shows the version of Go as well as the platform and CPU architecture of the container/host. One thing to note is that both examples use the exact same **docker container run** command. We do not have to tell Docker that we need the Linux x64 or Windows x64 versions of the image. We just run normal commands, and let Docker take care of getting the right image for the platform and architecture we are running!

Linux on x64 example

```
1 $ docker container run --rm golang go version
2 <Snip>
3 go version go1.14.2 linux/amd64
```



Got any feedback? Get in touch with us.



Windows on x64 example

```
1 > docker container run --rm golang go version
2 <Snip>
3 go version go1.14.2 windows/amd64
```



The Windows Golang image is currently over 5GB in size and may take a long time to download.

Docker manifest

The `docker manifest` command lets you inspect the manifest list of any image on Docker Hub. The following example inspects the manifest list on Docker Hub for the `golang` image. You can see that Linux and Windows are supported on various CPU architectures. You can run the same command without the `grep` filter to see the full JSON manifest list.

```
1 $ docker manifest inspect golang | grep 'architecture\|os'
```

All official images have manifest lists.

You can create your own builds for different platforms and architectures with `docker buildx`, and then use `docker manifest create` to create your own manifest lists.

The following command builds an image for ARMv7 called `myimage:arm-v7` from the contents of the current directory. It's based on the code in <https://github.com/nigelpoulton/psweb>.

```
1 $ docker buildx build --platform linux/arm/v7 -t myimage:arm-v7 .
```

You will be unable to run this command in the playground below due to platform limitations.

Got any feedback? Get in touch with us.

The beauty of the command is that you don't have to run it from an ARMv7 Docker node. In fact, the example shown ran on Linux on x64 hardware.

At the time of writing, `buildx` is an experimental feature and requires `experimental=true` setting in your `~/.docker/config.json` file as follows.

```
1 {
2   "experimental": true
```

```
2 export DOCKER_CLI_EXPERIMENTAL=enabled
3 }
```

You can also enable it through your CLI with the following command:

```
1 export DOCKER_CLI_EXPERIMENTAL=enabled
```

Playground

Terminal 1



Terminal



Click to Connect...

← Back lesson

☒ Mark As Completed

Next →

Pulling Images by Digest

Deleting Images

Got any feedback? Get in touch with us.

